IT HAPPENS @ RA ALE #3

Trending Content Prediction

Build a CSV/API → Feature Pipeline → Classification Model → Trend Score Service

METADATA IN, TREND SCORE OUT

STUDENT HANDOUT

PART 0

What This Project Actually Is

0.1 The problem in plain language

Platforms receive new uploads constantly — a short film from an independent creator, a feature from a studio partner, a clip from a channel that has never trended before. Somebody has to decide what to promote, and right now that decision is mostly gut feeling: a person scans new uploads, eyeballs early view counts, and guesses which ones deserve a push.

Metadata about a piece of content is secretly predictive: genre, duration, upload time, language and tags all correlate with what has trended before. A classification model is built for exactly that shape — learn the pattern from historical trending and non-trending content, then score a new upload against it. That is why this lands as a supervised ML problem and not a dashboard of manual rules.

Your job: build a pipeline that takes an uploaded item's metadata in the front door and returns a trending-likelihood score out the back door — with nothing manual in between.

0.2 Problem Statement

Develop a Machine Learning application that predicts whether a newly uploaded movie or short film is likely to become trending, based on attributes such as genre, duration, upload time, language, tags, and other available metadata.

0.3 Objective

The objective is to develop a classification model that identifies the popularity potential of uploaded content. The prediction should help content creators and OTT platforms understand which content is more likely to gain audience attention.

0.4 What makes this different

Most student ML exercises end like this: open a notebook, run .fit() once, print an accuracy number, close the tab. That is homework. Nobody outside that room can use it, and it only runs when you press Run.

This project does not end there. You must deliver:

- A running service. Someone must be able to submit metadata for content they have never shown you before, and get a trending-likelihood score back — automatically, with nobody from your team present.
- Something another team can start on their own machine, with one command, without phoning you for help. This is why the project uses a containerised, reproducible setup.
- Proof that it works. You must answer "does the model actually know what it's talking about?" with evaluation metrics, not a feeling.

There are two classic ways to fail this, and both are common in real companies:

- A model with no serving layer is a notebook nobody can query — the training happened, and getting a prediction out is now somebody's problem.

■ A model with no honest evaluation is a coin flip wearing a lab coat. It will produce a confident score whether or not the underlying pattern is real. That is worse than useless — it wastes a creator's or platform's promotion budget, quietly.

You are marked on the whole path, front door to back door.

PART 1

Important: The Model Must Be Honest

> **■ READ THIS SECTION CAREFULLY — Teams lose marks every time by getting this wrong.**

Your model can be as simple as a Logistic Regression baseline on hand-picked features, or, if your team wants and your organisers allow it, a tree ensemble (Random Forest, XGBoost/LightGBM) or a model that also uses text embeddings from titles and tags. Either approach is acceptable.

What is not acceptable is a model that reports a confident trending prediction with no evaluation behind it. If the model has not genuinely learned a pattern that generalises to unseen uploads, the correct output is a low-confidence score paired with honest metrics — not a plausible-sounding "yes, this will trend."

Why this matters

- Trust — a confidently wrong "this will trend" recommendation wastes a creator's or platform's promotion budget, because people act on it.
- Verifiability — a judge (or a real user) must be able to check the prediction against held-out data and a reported metric, not just take the model's word for it.
- Class imbalance is real — trending content is rare. A model that always predicts "not trending" can score 95% accuracy and be completely useless. It costs nothing to report Precision/Recall honestly; a hallucinated accuracy number is a broken product next week.
- The point of the exercise — fitting any classifier to any CSV is a five-minute task. Building something that is evaluated honestly, handles class imbalance, and generalises to unseen uploads is the skill being taught.

If you use an LLM anywhere (e.g. to generate features from free-text tags/descriptions), you must still report the underlying classifier's metrics separately — an LLM's fluent explanation is not a substitute for a measured Precision/Recall/F1/ROC-AUC. See the contract in Part 4.

Ask your organisers whether an LLM or external API key is permitted at this event before assuming you can call one.

PART 2

Before the Session

2.1 Install and verify your environment

Install Python 3.11 (or your team's chosen language/runtime) and Docker Desktop / Engine. Then actually run something — do not just check that it is installed:

```
docker run --rm hello-world
python3 -c "import sklearn, pandas; print('ok')"
```

If either command fails, fix it before the session. There is no recovery time in the schedule.

2.2 Pre-pull / pre-install the heavy pieces

scikit-learn, pandas, and (if used) xgboost/lightgbm should be installed at home, on a good connection:

```
pip install pandas scikit-learn xgboost fastapi uvicorn joblib
```

2.3 Bring test data

The task must work against metadata your pipeline has not been hand-tuned against in advance. Prepare at home so you are not scrambling on the night:

- A small, clean labelled sample (50–100 rows) to sanity-check the pipeline end to end.
- A larger sample (thousands of rows) to see how training and evaluation behave at real volume.
- A deliberately broken sample — missing fields, an unseen genre value, a null upload time — to test that your service fails politely instead of crashing.

2.4 Datasets — availability confirmed

Yes, public datasets are available that closely match the required attributes (genre, duration, language, tags, upload time, engagement). No single dataset is a perfect one-to-one match for "OTT short film uploads," so combine a video/engagement dataset (trending label + upload-time features) with a movie-metadata dataset (richer genre/language/cast attributes).

Trending YouTube Video Dataset (datasnaek/youtube-new)

https://www.kaggle.com/datasets/datasnaek/youtube-new

Several months of daily trending videos across US, GB, DE, CA, FR, RU, MX, KR, JP and IN, up to 200 trending videos per day per region. Closest public proxy for "trending short-form content," and already includes a natural trending label (appearance on the daily trending list).

Relevant fields: video title, channel title, publish (upload) time, tags, category, views, likes/dislikes, comment count, description, trending date/region

YouTube Trending Video Dataset — updated daily (rsrishav)

https://www.kaggle.com/datasets/rsrishav/youtube-trending-video-dataset

A more recent, continuously updated version of the trending-video dataset above; useful for a longer or fresher time span.

Relevant fields: video/channel metadata, publish time, tags, category, engagement stats, trending date/region

TMDB 20000 Trending Movies Dataset (yasirabdaali)

https://www.kaggle.com/datasets/yasirabdaali/tmdb-20000-trending-movies-dataset

~20,000 movies that have trended on TMDB — genre/language/runtime features on the movie side.

Relevant fields: title, genre(s), original language, runtime, release date, popularity score, vote average/count

TMDB Movie Popularity Prediction dataset (Kaggle competition)

https://www.kaggle.com/competitions/movie-popularity-prediction/data

~45,000 movies (MovieLens/TMDB merge, released on or before July 2017) framed explicitly as a popularity-prediction task — a good structural template.

Relevant fields: genre, runtime, language, cast/crew, budget, popularity score

TMDB + IMDB Merged Movies Dataset (ggtejas)

https://www.kaggle.com/datasets/ggtejas/tmdb-imdb-merged-movies-dataset

~400K TMDB records enriched with IMDb non-commercial data and director/writer/cast fields.

Relevant fields: id, title, vote_average, vote_count, status, release_date, revenue, runtime, adult flag, genres, directors, writers, cast

Suggested combination: use the YouTube trending dataset as the primary source (it already has an upload-time field and a genuine trending label); use the TMDB datasets to enrich genre/language/runtime features, or to train a second model for feature-length films.

2.5 APIs — needed, and confirmed available

Yes, a live API is recommended, mainly to (a) enrich static datasets with up-to-date metadata, and (b) fetch metadata for genuinely new uploads at inference time, since the datasets above are historical snapshots.

Detail	Value
Primary API	The Movie Database (TMDB) API — https://api.themoviedb.org/3/
Trending endpoint	GET /trending/{media_type}/{time_window} — e.g. /trending/movie/day or /trending/movie/week
Auth	Free developer API key (v3) or read-access token (v4); request under Account Settings -> API at themovied
Cost	Free for non-commercial use with TMDB attribution; commercial use needs a written agreement
Rate limit	~40 requests/second (soft limit) per API key
Alternatives	YouTube Data API v3 (free tier, quota-limited), OMDb API (free key), TVmaze API (keyless)
Docs	https://developer.themoviedb.org/reference/getting-started

Ask your organisers whether external API calls are permitted at this event before assuming you can make them during the build window.

2.6 Your credentials

Store any TMDB / YouTube / OMDb API key as an environment variable in your compose file or .env — never hard-code it into source.

2.7 A stack that fits the clock

Layer	Suggested	Note
Language	Python 3.11	Mature pandas / scikit-learn / xgboost ecosystem
UI	A single lightweight page (plain HTML -- fetch, no build step)	A working ugly UI beats a broken pretty one
Feature store / cache	Local Parquet/CSV or SQLite	No separate service needed for this scale
Model	Logistic Regression baseline, then RandomForest, LightGBM or XGBoost	Start simple; add complexity that measurably helps
Serving API	FastAPI or Flask	Validates request bodies; async support helps with API enrichment
Persistence	joblib / pickle for the trained model, versioned by filename	Never overwrite a model file silently — keep the old one until the ne

Three notes worth knowing before you start: an external API can rate-limit or time out — your enrichment step must retry or fall back gracefully, not crash. A model file that fails to load must produce a clear /health failure, not a silent wrong answer. pip install --no-cache-dir saves real space for free; keep the training image and the serving image separate and minimal.

PART 3

What You Are Building

Five moving parts, controlled by one pipeline script or docker-compose.yml file.

```
Browser / Client
submit content metadata (genre, duration, upload time, language, tags)
| ^
POST /predict answer + confidence + metrics
v |
```

```
api
/train . /status . /predict . /health
| ^
reads training data reads/writes model artifact
v |
feature-pipeline
cleans + encodes metadata, joins API enrichment
|
v
model-store
versioned classifier artifact (joblib/pickle) + metrics.json
```

- **ui** — lets a user enter or upload content metadata, preview what was received, and view the returned trend score with its supporting metrics.
- **api** — one backend, four jobs: retrain or refresh the model (POST /train), report training progress (GET /status), score new content (POST /predict), and report readiness (GET /health).
- **feature-pipeline** — cleans raw metadata, encodes categorical fields, engineers upload-time features, and optionally enriches records via the TMDB/YouTube API. This decouples "receiving raw metadata" from "a trained, servable model," so a slow enrichment call never blocks scoring of content that does not need it.
- **model-store** — the trained classifier artifact, versioned, queried by the api for /predict and re-written by /train.

Why separate the feature pipeline from the API's request handler?

- The /predict endpoint returns quickly instead of blocking on however long an external API enrichment call takes.
- If the enrichment API is briefly unreachable, the request can fall back to metadata-only features instead of failing outright.
- You can re-run feature engineering and retrain without asking the user to resubmit anything.

This is exactly how ML serving pipelines are built at real companies — feature computation, model training, and model serving are deployed, scaled, and restarted independently.

PART 4

The Interface Contract

Your API must accept and return these shapes. Other teams' tests and the judges' checks depend on it.

TRAIN

```
POST /train
Content-Type: application/json (or multipart/form-data with a CSV file)

-> 202 Accepted
{
  "job_id": "t3f1",
  "rows_received": 5000,
  "status": "queued"
}
```

STATUS

```
GET /status?job_id=t3f1

{
```

```
  "job_id": "t3f1",
  "status": "training", // queued | training | complete | failed
  "rows_total": 5000,
  "rows_processed": 3200,
  "rows_failed": 4
}
```

HEALTHCHECK

```
GET /health

{
  "status": "ok",
  "model_loaded": true,
  "enrichment_api_connected": true
}
```

/health must report anything other than ok until a valid trained model is genuinely loaded — not merely "the container has started."

PREDICT

```
POST /predict
{
  "title": "Midnight Run",
  "genre": "Thriller",
  "duration_minutes": 24,
  "upload_time": "2026-08-06T18:30:00Z",
  "language": "en",
  "tags": ["suspense", "indie", "short-film"]
}

-> 200 OK
{
  "trending_probability": 0.71,
  "predicted_label": "trending",
  "model_version": "rf_v3",
  "top_features": ["genre=Thriller", "upload_hour=18", "tag_count=3"],
  "grounded": true
}
```

If the model cannot confidently score the input (e.g. an unseen category with no fallback encoding), grounded must be false and predicted_label must say so plainly — never fabricate a confident score. See Part 1.

PART 5

The Reproducibility Rule (Mandatory)

Because the input metadata is dynamic — you do not know every category value ahead of time — keep the feature encoding generic and versioned:

```
raw_metadata --> feature_pipeline (fixed schema + versioned encoders) --> model_vN.joblib
+ metrics_vN.json
```

Every training run must fix its random seed and record the exact train/validation/test split used — never retrain silently over the previous model. If your pipeline does not version its encoders, retraining on a new CSV or replaying the same data will silently change what "genre=Thriller" means to the model between runs.

> ■ **NO PARTIAL CREDIT ON THIS ONE — Two clean training runs against the same dataset and seed must produce the same evaluation metrics (within floating-point tolerance). Producing different metrics on a second run because of an unfixed seed or unversioned split scores zero on the reproducibility component of Part 11 — no partial credit.**

If you subsample or transform rows before training, do it inside the feature pipeline — never mutate the source file in ./data.

PART 6

How to Actually Do This

Read this whole part before coding. It exists so you do not lose an hour to a trap we already know about.

6.1 Keep the feature set boring

Genre, duration, upload-time features, language, and tag count is enough to pass. You are marked on the system, not on a beautifully engineered feature set. If you finish early, then enrich the model — pull real popularity signals from the TMDB API, add text embeddings from titles — but keep the simple feature set working as your fallback.

6.2 Build the pipe before the model

This is the single most important paragraph in the document. The most common way to fail is to have a clever model at 7:40 and no working serving endpoint. Build the plumbing with fake logic first, then fill it in.

So in your first block of time, build the skeleton with fake logic:

- ui submits any metadata and shows "submitted"
- api's /train accepts a CSV and writes a dummy metrics file, then returns a fake job_id
- api's /status returns a hardcoded "complete"
- api's /predict returns a hardcoded canned score with grounded: false

Once your pipeline runs end to end, replace the dummies one at a time. From that moment onward, every minute you spend improves something that already works — and if you run out of time, you still have a working system.

6.3 "Trained" is not "ready"

A model that finished .fit() is not the same as a model that is safe to serve — it must also pass a sanity evaluation. Fix it properly: /health should only report ok once a model has been trained, loaded, and its metrics have cleared a minimum bar you define. A retry/reload loop inside your own serving code is an acceptable alternative — but say in your report which you chose and why.

6.4 Keep credentials out of the image

Any TMDB/YouTube/OMDb API key goes into environment variables, never into a Dockerfile line or a hard-coded string in your source. Be ready to explain why: an image with a baked-in key can leak the key anywhere the image is pushed, and rotating the key means rebuilding the image.

6.5 Status will lie to you if you let it

It is tempting to mark training complete once every row has been read. That is not the same as every row being safely featurised and used — a row can fail to parse. Track rows_failed honestly, and do not report complete until processed + failed accounts for every row that was received.

6.6 Make it repeatable

Fix your random seed and your split logic so a rerun on the same dataset produces the same metrics. Judges will retrain your pipeline on your test dataset and compare metrics against your report. They must match.

6.7 Assume hostile input

Real services receive garbage. Before the freeze, throw these at your API and make sure it responds sensibly instead of crashing:

- an empty training CSV
- a CSV with a header row but zero data rows
- a /predict request missing a required field
- a /predict request with a genre or language never seen during training
- a /predict request sent before any model has been trained

A clean error message is a pass. A stack trace, a dead container, or a confidently wrong trend score is not.

PART 7
Requirements

7.1 Must-have — this is your pass

#	Requirement
1	A single command brings up ui, api and the feature/model pipeline — with zero manual steps
2	Raw metadata reaches the trained model only via the versioned feature pipeline, never scored from an unencoded raw field
3	Every training run fixes its random seed and split — no metric drift on a second run of the same data
4	GET /health reports not-ok until a valid trained model is genuinely loaded
5	GET /status reports queued, training, complete or failed with real row counts, not a hardcoded value
6	POST /predict follows the contract in Part 4, including model_version and grounded on every answer
7	A prediction request the model cannot confidently score returns grounded: false and says so, instead of guessing
8	All dependency versions pinned — no unpinned "latest" installs
9	Containers/processes run as a non-root user
10	Two clean training runs against the same dataset produce identical evaluation metrics

7.2 Stretch — this is how you win

- Serving image under 400 MB
- p95 /predict response time under 200 ms, actually measured, with the method stated
- The ui shows live progress while a large dataset is still training, not just a spinner
- Survives every item in the hostile-input list in 6.7
- The pipeline detects likely useful text signal in titles/tags (e.g. TF-IDF) and shows it improves metrics over the metadata-only baseline
- A second training run on the same dataset skips redundant feature recomputation
- Model artifacts are versioned so a bad retrain can be rolled back without retraining

PART 8

Timeline

Time	Phase	You are done when
5:00 – 5:10	Read, agree the architecture, split the work	Everyone knows what they own
5:10 – 5:35	Skeleton — pipeline, dummy logic end to end	One command runs ui -> api -> feature-pipeline -> model-store
5:35 – 6:00	Real ui — submit metadata, preview, view score	A realistic metadata record can be submitted and previewed
6:00 – 6:30	Real feature pipeline + training — rows ready, model trained	A held-out evaluation shows real, non-hardcoded metrics
6:30 – 7:00	Real api — /status reports true progress and /health is honest	curl /status matches what's actually been processed
7:00 – 7:20	Real predictions — grounded scores with real contributions	A realistic record gets a correct, grounded trend score
7:20 – 7:25	Hardening — non-root, pinned deps, host input	The Part 10 checklist passes
7:25 – 7:30	BUILD FREEZE — commit and push	Nothing further is edited
7:30 – 8:00	Report	REPORT.md committed

Nominate one person as timekeeper and have them call out each checkpoint aloud.

> ■ **THE 7:25 FREEZE IS HARD — Work that is not committed does not exist.**

PART 9

The Report (30 minutes, 30 marks)

One file: REPORT.md, in your repository. Plain language. These sections, in this order.

9.1 What we built

Five sentences plus the architecture. State honestly what works and what does not. An accurate account of a half-working pipeline scores far above an optimistic account of the same pipeline.

9.2 The data and the feature model

The datasets you actually trained with, row counts, class balance (trending vs not), and the exact features your pipeline produces. If you enriched via an external API, explain what you fetched and what you did when the API was unavailable.

9.3 Methods

Fill this in — one line per row:

Decision	Chosen	Rejected	Reason
Trending label definition			
Model type			
Handling class imbalance			
How api knows model is ready			

We are checking whether you chose or merely defaulted.

9.4 Results

A table of at least eight content items you actually scored, whether the prediction was correct and grounded, and — most important — your explanation of why the ones that failed, failed. That paragraph carries more marks than the table above it.

Content item	Prediction given	Correct?	Grounded?

9.5 How we worked

Who owned what, and whether that changed. Planned versus actual at each checkpoint in Part 8. Then two decisions, in this form:

Decision: what you decided — Options considered: what else was on the table — Chosen because: your reasoning — Cost accepted: what it cost you — Would revisit if: what would change your mind

And one dead end — something you tried, when you abandoned it, and what told you to stop. Knowing when to stop is a marked skill.

9.6 Limitations and next steps

What breaks in production. Be specific. "Add monitoring" earns nothing. "We have no way to detect concept drift when trending patterns shift with the season" earns a lot.

9.7 How to run it

Exact commands. A judge must be able to run your project from this section alone, with no verbal help from you.

P A R T 10
Pre-Freeze Checklist

At 7:15, stop adding features and verify every line.

- ■ Fresh training run from a clean environment works
- ■ No unpinned "latest" dependency anywhere
- ■ Any API key comes from an environment variable, not hard-coded
- ■ Processes/containers run as non-root
- ■ /health reports not-ok before a valid model is genuinely loaded
- ■ /status reports real row counts, including failures, not a hardcoded value
- ■ API survives an empty dataset, a malformed row, and a prediction request sent before training
- ■ A second training run on the same dataset does not change the reported metrics
- ■ Predictions include the model version and confidence, and say so honestly when ungrounded
- ■ REPORT.md committed
- ■ Everything pushed

P A R T 11
Marking

Component	Marks
Pipeline runs clean, first time, no manual steps	15
Correct architecture — feature pipeline decouples raw metadata from serving	10
Healthcheck and correct startup/readiness ordering (model load, API)	8

Hygiene — pinned dependencies, non-root, credentials via env, sane image/artifact sizes	7
UI works end to end — submit, preview, and prediction all function in the browser	5
API correctness and input handling	10
Reproducible training — reruns do not change reported metrics	10
Model honesty — predictions grounded in real evaluation, honestly says "I don't know" when unconfident	10
Report: methods and decisions	12
Report: results interpretation	8
Report: process and honesty	5
Total	**100**

Read that distribution carefully. Model cleverness is worth 10 marks. Engineering and reasoning are worth 90.

A team with a simple Logistic Regression baseline, a clean pipeline and a sharp report will beat a team with an XGBoost ensemble and a broken serving endpoint. That is not an accident — it is the point of the exercise.

P A R T 12

Frequently Asked

Do we need internet during the hackathon?

Only if you plan to call the TMDB/YouTube API live for enrichment. Pre-install your Python dependencies beforehand — see Part 2.2.

Do we need a fixed dataset ahead of time?

No. The task must work against dynamic metadata. Bring your own test samples — see Part 2.3. Public datasets are listed in Part 2.4 if you need a starting point.

Can we use any programming language or ML library?

Yes. Python with scikit-learn is the path of least resistance, but nothing in the requirements demands it.

Do we need a separate feature-store service?

No. A local Parquet/CSV or SQLite file is enough at this scale and boots faster than standing up a dedicated service.

Can our pipeline call a language model?

Ask your organisers for the rule in force at this event. If permitted, the model may only help engineer text features (e.g. summarising tags) — it must not replace the classifier's own evaluated prediction. See Part 1.

What if training crashes partway through a large dataset?

/status must report failed and the row count actually processed. A silent crash that leaves /status saying training forever is treated as a failed run.

Can we skip the ui and just prove it with curl?

Yes, if you're short on time — a working feature-pipeline -> model -> /predict flow with an honest report beats a broken system with a pretty front end every time.

Our model gets a lot of predictions wrong. Are we finished?

No. Model correctness is only 10 of 100 marks. A model that honestly reports low confidence, sitting on a clean and well-evaluated pipeline, scores well. Report the weak results honestly — that explanation earns marks.

Good luck — and build the pipe first.